

Justificación de Decisiones de Diseño y Arquitectura

1. Uso de Patrones de Comportamiento (Strategy y Factory)

Para resolver la simulación de lecturas de los sensores (Apartado 2), se ha optado por implementar el patrón de diseño Strategy mediante la interfaz IEstrategia. Esta decisión nos permite extraer la lógica matemática de generación de datos fuera de la clase del sensor. De este modo, podemos cambiar el comportamiento de un sensor en tiempo de ejecución sin necesidad de modificar su código interno, respetando el principio de Responsabilidad Única (SRP). Adicionalmente, la instanciación de los sensores se ha centralizado en la enumeración TipoSensor, la cual actúa como una pequeña factoría (Factory Method), ocultando la complejidad de la creación de objetos a la estación meteorológica.

2. Flexibilidad en el Procesamiento de Datos (Conversores Concatenados)

El sistema de conversión de unidades se ha diseñado con un alto grado de desacoplamiento utilizando la interfaz IConversor. En lugar de codificar las transformaciones matemáticas (Celsius a Kelvin, etc.) dentro de los sensores, se ha delegado en procesadores inyectables. El punto clave de este diseño es la clase ConversorConcatenado, que permite anidar múltiples conversores compatibles. Gracias a esto, el sistema es infinitamente extensible: si en el futuro se añade una nueva unidad, solo hay que crear su conversor específico, y el sistema podrá triangular conversiones complejas sin alterar el código existente.

3. Jerarquía de Excepciones como Lógica de Negocio

El sistema de alertas (Apartado 4) se ha modelado mediante una jerarquía de excepciones propias encapsuladas en los paquetes excepciones y alertas. Se ha dividido conceptualmente en dos ramas: los fallos críticos que bloquean el sensor (SensorSinCalibrarException) y los avisos informativos (CambioBruscoException). Al hacer que el método medir() lance estas excepciones, delegamos la responsabilidad de registrar el fallo a la Estación Meteorológica, permitiendo que el sensor sea una entidad "ciega" que solo se preocupa por sus matemáticas, logrando una cohesión máxima.

4. Desacoplamiento de la Presentación

Para el sistema de visualización (Apartado 5), se ha evitado por completo que la Estación Meteorológica sepa sobre HTML o Markdown. Por otro lado, la interfaz IFormateador consume este documento. Una decisión de diseño fundamental ha sido utilizar un método default en la interfaz IFormateador para centralizar la lógica de entrada/salida de ficheros. Esto evita duplicar el código de creación de archivos en FormateadorHTML y FormateadorMarkdown.

Diagrama de clases final:

